book-qr-manager 学習ノート

学習日：2026年9月6日
テーマ：Location取得・更新・削除APIとCRUDの完成

1. 今日やったこと

前回完成したLocation登録API（POST）の続きとして、取得・更新・削除APIを実装しました。

操作 HTTPメソッド URL 内容

Create POST `/locations` Locationを登録 Read GET `/locations` 一覧取得 Read GET `/locations/:id` ID指定で1件取得 Update PATCH `/locations/:id` 一部更新 Delete DELETE `/locations/:id` 削除

2. 一覧取得：GET /locations

Serviceに次の処理を追加しました。

```
async findAll(): Promise<Location[]> {
  return this.locationRepository.find();
}
```

`Repository.find()`は対象Entityのデータを一覧取得します。複数件を返すため、型は`Location[]`です。非同期処理なのでメソッド全体の戻り値は`Promise<Location[]>`になります。

Controllerでは次のように呼び出します。

```
@Get()
async findAll(): Promise<Location[]> {
  return this.locationsService.findAll();
}
```

流れは次のとおりです。

```
GET /locations
  ↓
LocationsController.findAll()
  ↓
LocationsService.findAll()
  ↓
locationRepository.find()
  ↓
PostgreSQL
```

```
  ↓
Location[]
```

Postmanで`200 OK`とLocationの配列が返ることを確認しました。

3. ID指定取得：GET /locations/:id

`GET /locations/3`のように、特定IDのLocationを取得するAPIを作成しました。

```
@Get(':id')
```

の`:id`はURLから値を受け取る部分で、**パスパラメータ**と呼びます。

Controllerでは`@Param()`で取得します。

```
@Get(':id')
async findOne(@Param('id') id: string): Promise<Location> {
  return this.locationsService.findOne(Number(id));
}
```

URLから取得した値は基本的に文字列なので、`Number(id)`で`number`へ変換してServiceへ渡します。

```
"3" (string)
  ↓
Number(id)
  ↓
3 (number)
```

4. findOneBy()と404処理

ServiceではTypeORMの`findOneBy()`を使いました。

```
async findOne(id: number): Promise<Location> {
  const location = await this.locationRepository.findOneBy({ id });

  if (!location) {
    throw new NotFoundException(`Location with ID ${id} not found`);
  }

  return location;
}
```

`findOneBy({ id })`は、条件に一致するEntityを1件検索します。見つからない場合は`null`になります。

そこでNotFoundExceptionを使い、存在しないIDならHTTPの404 Not Foundを返すようにしました。

```
findOneBy({ id: 99 })
  ↓
null
  ↓
if (!location)
  ↓
NotFoundException
  ↓
404 Not Found
```

Postmanでも存在しないIDに対して404になることを確認しました。

5. UpdateLocationDto

部分更新用にupdate-location.dto.tsを作成しました。

```
export class UpdateLocationDto {
  name?: string;
  shelf?: string;
  row?: string;
  column?: string;
  notes?: string;
}
```

PATCHでは変更したい項目だけ送信できるため、すべて任意項目（?）にしています。

```
CreateLocationDto
→ nameは必須

UpdateLocationDto
→ すべて任意
```

6. 更新：PATCH /locations/:id

Serviceでは次の処理を作りました。

```
async update(
  id: number,
  updateLocationDto: UpdateLocationDto,
): Promise<Location> {
  const location = await this.findOne(id);
  Object.assign(location, updateLocationDto);
```

```
    return this.locationRepository.save(location);
  }
```

まず`findOne(id)`で既存Locationを取得します。存在しなければ、既存の404処理をそのまま再利用できます。

`Object.assign(location, updateLocationDto)`は、送られてきた更新項目だけを既存Locationへ上書きします。

例えば`row`だけ送れば、`name`や`shelf`は変更されません。

最後に`save()`で保存します。

```
新しいEntity → save() → INSERT
既存Entity   → save() → 更新
```

Controllerは次の形です。

```
@Patch(':id')
async update(
  @Param('id') id: string,
  @Body() updateLocationDto: UpdateLocationDto,
): Promise<Location> {
  return this.locationsService.update(Number(id), updateLocationDto);
}
```

Postmanから`row`を「2段目」から「3段目」へ変更し、pgAdminでも更新を確認しました。

7. 削除：DELETE /locations/:id

Serviceでは、

```
async remove(id: number): Promise<void> {
  const location = await this.findOne(id);
  await this.locationRepository.remove(location);
}
```

としました。

`findOne(id)`で削除対象を取得してから、`Repository.remove()`でDBから削除します。ここでも存在しないIDなら`findOne()`の404処理を再利用できます。

Controllerは、

```
@Delete(':id')
async remove(@Param('id') id: string): Promise<void> {
  return this.locationsService.remove(Number(id));
}
```

です。

Postmanから`DELETE /locations/5`を実行し、pgAdminでid=5が削除されていることを確認しました。

8. CRUDとは？

CRUDはデータ操作の基本となる4種類です。

```
C = Create → 作成
R = Read   → 取得
U = Update → 更新
D = Delete → 削除
```

今回Locationで一通り実装できました。

```
POST   /locations      → Create
GET    /locations      → Read（一覧）
GET    /locations/:id  → Read（1件）
PATCH  /locations/:id  → Update
DELETE /locations/:id  → Delete
```

9. Location CRUDの全体像

```
HTTP Request
     ↓
LocationsController
     ↓
LocationsService
     ↓
Location Repository
     ↓
PostgreSQL
```

各Repositoryメソッドとの対応は、

```
POST
→ create()
→ repository.create()
```

```
→ repository.save()

GET（一覧）
→ findAll()
→ repository.find()

GET（1件）
→ findOne()
→ repository.findOneBy()

PATCH
→ update()
→ findOne()
→ Object.assign()
→ repository.save()

DELETE
→ remove()
→ findOne()
→ repository.remove()
```

です。

10. 今日の重要ポイント

- `find()`：複数のEntityを取得する
- `findOneBy()`：条件に一致するEntityを1件取得する
- `@Param('id')`：URLのパスパラメータを取得する
- `NotFoundException`：対象が存在しない場合に404を返す
- `Object.assign()`：既存オブジェクトへ更新値を上書きする
- `save()`：新規登録だけでなく既存Entityの更新にも使える
- `remove()`：取得済みEntityをDBから削除する

11. 次回から何をする？

次回は**Book側のAPI実装**へ進みます。

LocationでCRUDを一通り経験したため、Bookでは同じパターンを復習しながら、自分で実装する範囲を少し増やします。

予定は次の流れです。

```
① Book用DTOを作成・確認
      ↓
② POST /books
   書籍登録
      ↓
③ GET /books
   書籍一覧取得
      ↓
```

```
④ GET /books/:id
   ID指定取得 + 404
      ↓
⑤ PATCH /books/:id
   書籍更新
      ↓
⑥ DELETE /books/:id
   書籍削除
```

BookにはLocationとのリレーションがあります。

```
Location 1
   │
   └──── Book N
```

Book側のCRUDを作りながら、`location_id`を使って「この本がどのLocationに置かれているか」を扱えるようにしていきます。

その先では、

```
本を登録
 ↓
Locationを紐付ける
 ↓
BookからLocation情報を取得
 ↓
QRコードからBookを特定
 ↓
本の情報・保管場所を表示
```

という、このアプリの中心機能につなげていく予定です。

復習用ひとこと

ControllerがHTTPリクエストを受け取り、Serviceが処理を担当し、RepositoryがPostgreSQLを操作する。

今回Location CRUDを完成させたことで、NestJSで基本的なAPIを作る一連の流れを一通り実装できました。